Using Kredis to Store Short-Term State

Sometimes you have information that you want to store for a finite amount of time and that you'd like to be able to access quickly. In our context, this might be about the state of the UI at any given time—we might use that information to redraw the screen—but it's not really business data and we don't necessarily want to store it in our regular database with our regular data. One example might be a list of a user's recent searches. Or a list of what parts of the UI are visible or not (in our application, we might want to persist whether the list of favorite concerts is visible or not).

Typically, this kind of ephemeral data is stored in a key/value storage tool like Redis. We've already installed Redis to support background jobs, but as a general-purpose key/value storage tool that lives in memory, it can be used to store all kinds of data. From a Rails perspective, a problem with using Redis is logistical—every bit of data is identified only by a unique key so managing the keys and pairing data with Rails objects can be a bit of work.

Which is where Kredis (short for Keyed Redis) comes in. Kredis[71] is a gem from the Rails core team that enables you to assign ActiveModel objects attributes that are stored in Redis that you can then treat mostly like any other attribute.

Let's go through a short example. In our existing concert page we have an edit button for each concert that flips the display of that concert to an edit form. Sadly, the state of that concert display is not persisted, so if you open the page in another tab, the concert is not open for edit anymore. What we're going to do is persist that information. Before you get too excited, we're not actually going to persist the partial edit (though you could do that as well), just check to see whether the concert is open for edit or not. So it's more a proof-of-concept and not a ship-to-production.

First, we need to install Kredis. The gem's already installed; it's been sitting in our Gemfile quietly this whole time waiting for this moment. But we do need to